

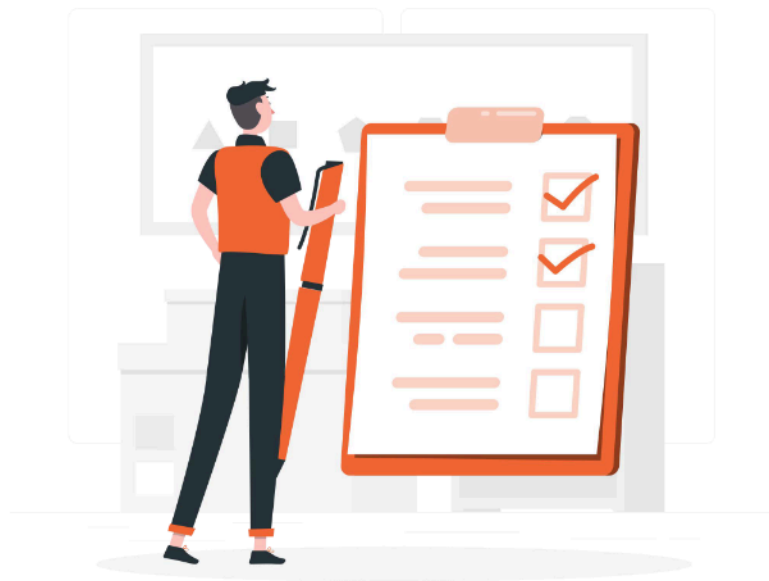
RAG Optimization: Re-ranking, Caching & Scaling

Name | Job Title



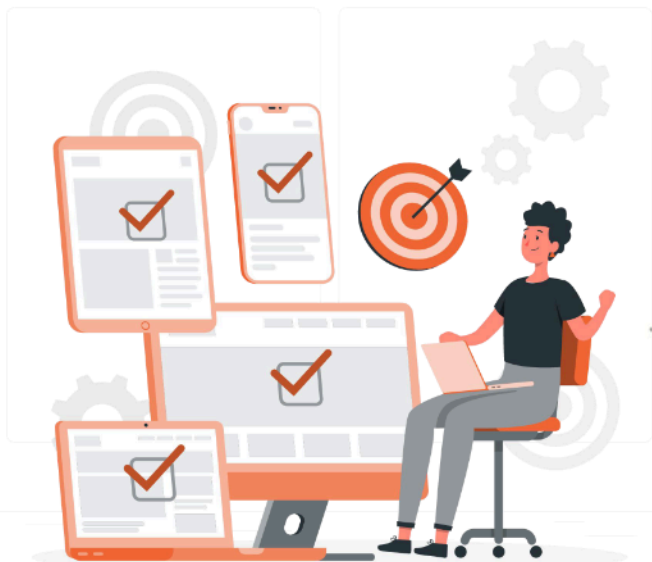
Today's Agenda

1. Re-ranker Fundamentals
2. Reducing Hallucination
3. Caching Strategies
4. Large Document Handling
5. RAG Evaluation Basics



Learning Outcome

1. Understand basic Re-ranker technique
2. Reducing Hallucination in RAG
3. Implement caching strategies
4. Handle large-scale documents
5. Apply RAG evaluation frameworks





Re-ranker Fundamentals

Vector Search Returns

Doc 1: Q2 revenue report (wrong quarter)

Doc 2: Q3 revenue summary ✓ relevant

Doc 3: Q3 headcount growth (wrong metric)

Doc 4: Revenue methodology (tangential)

Doc 5: Q3 financial highlights ✓ relevant

LLM (Azure OpenAI API)

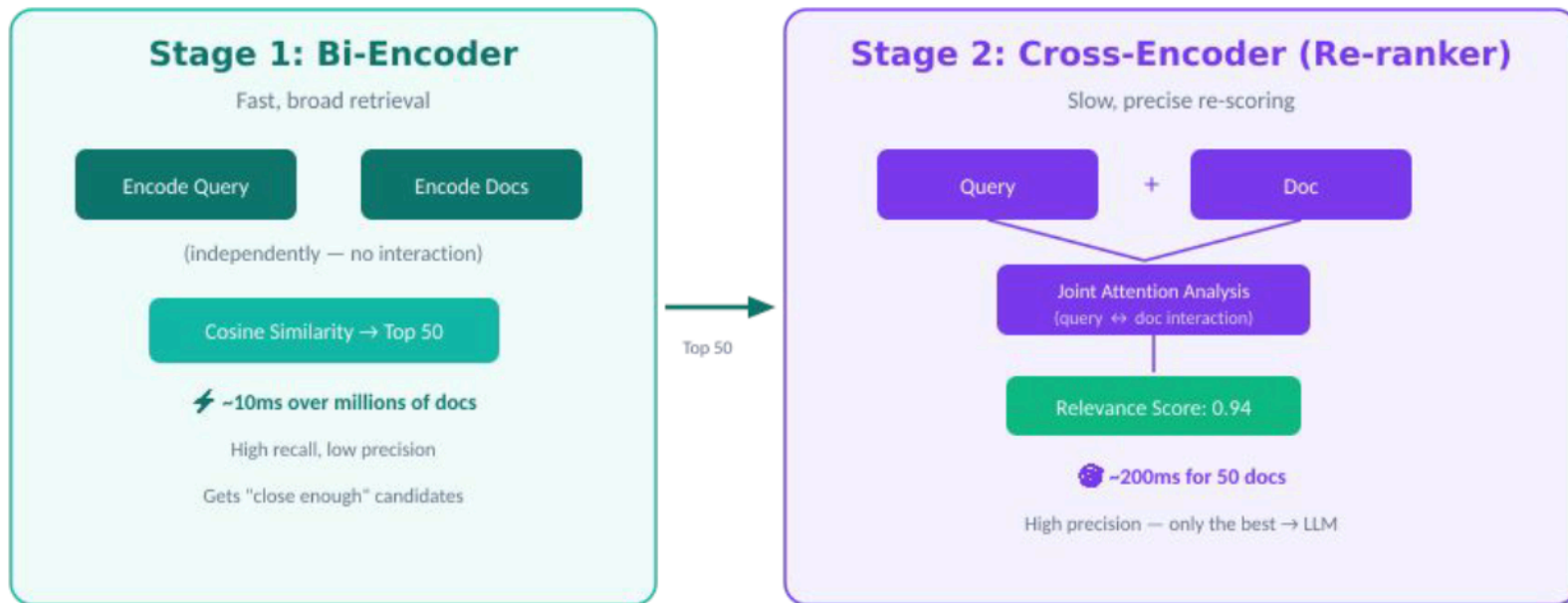
⚠ Noisy context → bad/hallucinated answer

Re-ranker Technique

The Problem: Why Simple Retrieval Fails

When a user asks a question, vector search returns the top-k documents that are semantically similar. But **"semantically similar" ≠ "actually relevant"**. The LLM then tries to generate an answer from this noisy context, leading to wrong or hallucinated responses.

The Solution: Two-Stage Retrieval

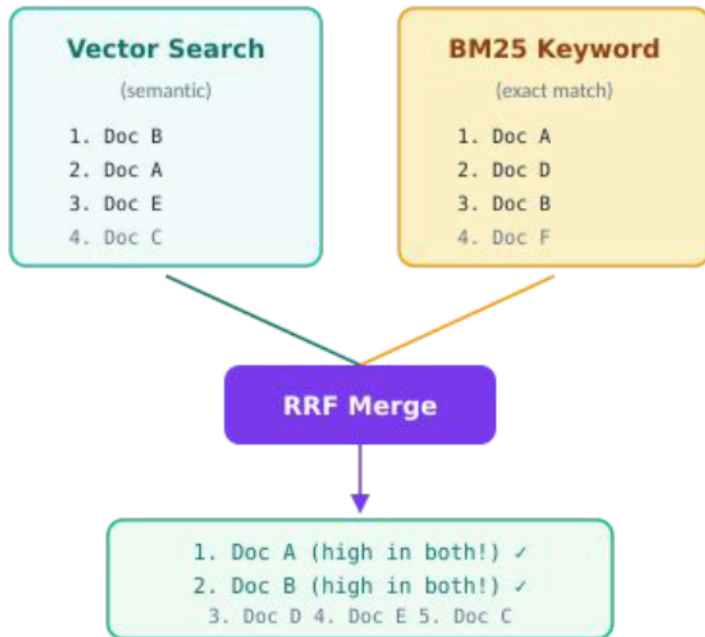


The Solution: Reciprocal Rank Fusion (RRF)

Vector search is great at meaning, but misses exact keywords. **Keyword search (BM25)** catches exact terms but misses synonyms.

RRF merges both result lists using a simple formula:
score = $\sum 1 / (k + \text{rank})$

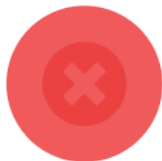
Documents ranked high in both lists get the best combined scores, capturing both semantic meaning and keyword precision.





Reducing Hallucination

Why RAG Hallucinations Happen



Missing Context

The retrieved chunks don't contain the answer at all — the LLM improvises from training data



Conflicting Context

Multiple retrieved docs say different things — the LLM picks one (or blends them incorrectly)



Poorly Formatted Context

Tables, lists, or multi-part docs get chunked badly — the LLM misinterprets fragments



Core insight: Hallucinations are a context problem, not just a model problem



Chain-of-Verification (CoVe)

Prompt the model to verify its own output



Reducing Hallucination

#1 Chain-of-Verification (CoVe)

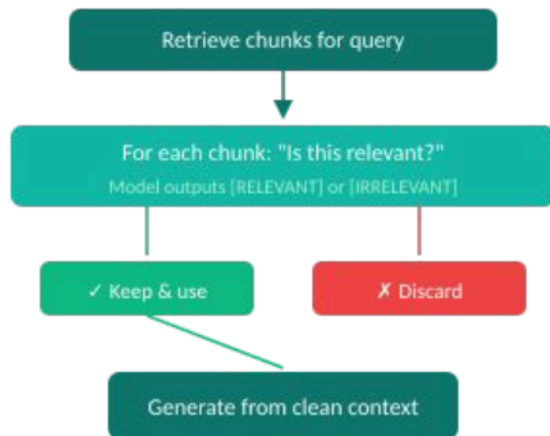
Chain-of-Verification (CoVe) is a **multi-stage prompting technique** where the model deliberates on its own output before finalizing it.

- Forces explicit fact-checking phases.
- Reduces error rates



Self-RAG

Model decides if context is useful BEFORE answering



Reducing Hallucination

#2 Self-RAG

Self-RAG introduces a "**self-critique**" mechanism. It uses specific **reflection tokens** to **analyze whether the retrieved documents are relevant**, if the generation is supported by those documents, and if the final response is actually useful.



Caching Strategies

Three Layers of Caching

~0ms

response time

Exact Match

Why: Identical queries happen more than you think — FAQs, onboarding, repeated reports.

How: Hash the query string → check Redis/Memcached → return cached response instantly.

~40%

fewer API calls

Semantic Cache

Why: "What's our Q3 revenue?" and "Q3 revenue numbers?" are different strings but the same question.

How: Embed the query → search a vector store of past Q-A pairs → if cosine sim ≥ 0.95 , return cached answer.

~50%

input token savings

Prompt Cache

Why: Your system message + reference docs are the same across many requests — why pay for them every time?

How: API-level caching of the system prompt prefix. Azure OpenAI and other providers support this.

Exact Match Caching

The Problem

Many users ask the exact same question — FAQ-style queries, onboarding questions, repeated reports. Every identical query triggers a full LLM call, wasting money and adding latency.

The Idea

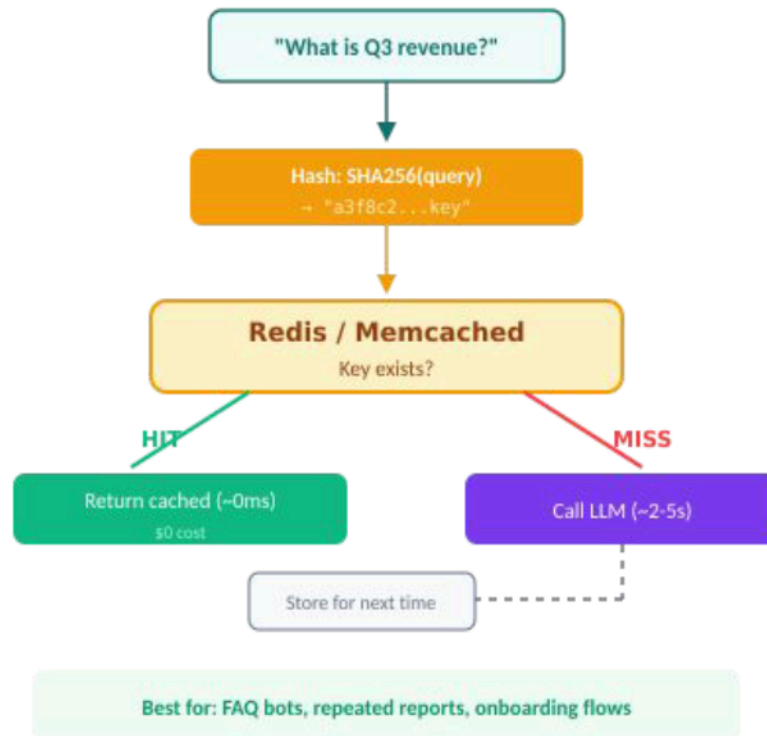
Hash the query string as a key. Store the LLM response as the value in Redis or Memcached. On the next identical query, return the cached response instantly — zero LLM cost.

Why it works:

Simple, deterministic, near-zero latency (~0ms). No embedding needed, no similarity search. Just a hash lookup.

Trade-off:

Only catches identical strings. "Q3 revenue?" and "What was Q3 revenue?" are cache misses. That's where semantic caching comes in.



Semantic Caching

The Problem with Exact Match

"What is Q3 revenue?" and "Q3 revenue numbers?" mean the same thing but have different hashes. Exact match misses all paraphrased queries.

The Idea (GPTCache Pattern)

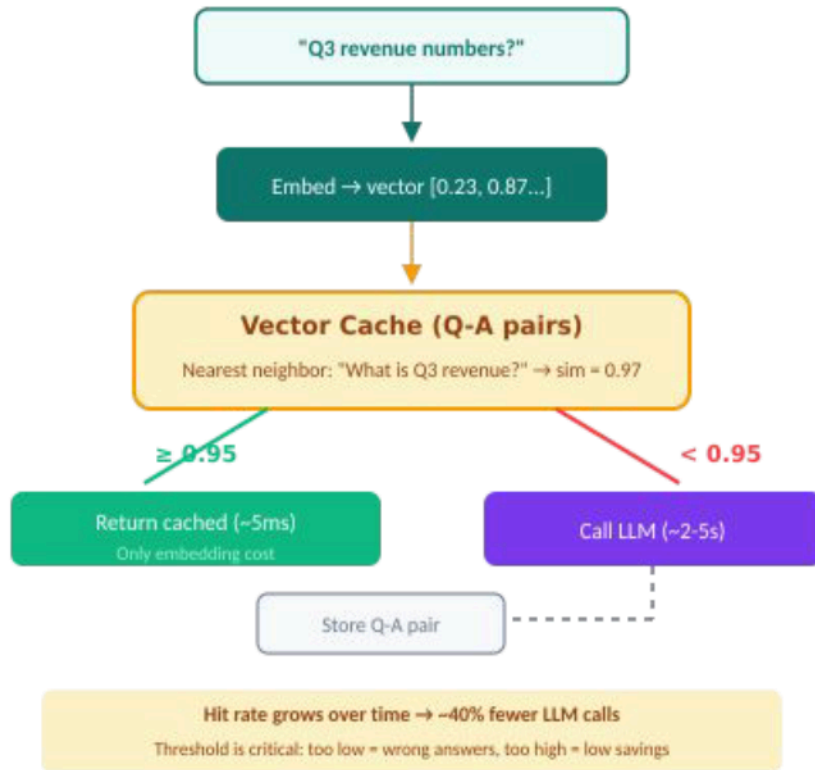
Embed the query into a vector. Search a vector store of previously asked queries. If the cosine similarity is above a threshold (e.g., 0.95), return the cached response instead of calling the LLM.

Why it works:

Paraphrased queries get caught. Over time, the cache grows and the hit rate increases — reducing API calls by ~40% in typical production workloads.

Trade-off:

Requires an embedding model call (~5ms). Threshold too low = stale/wrong answers. Threshold too high = low hit rate. Tune carefully.



Prompt Caching

The Problem

In a typical RAG setup, every API call sends the same system message + the same reference documents + only a small user query. You're paying for thousands of input tokens that never change between requests.

The Idea

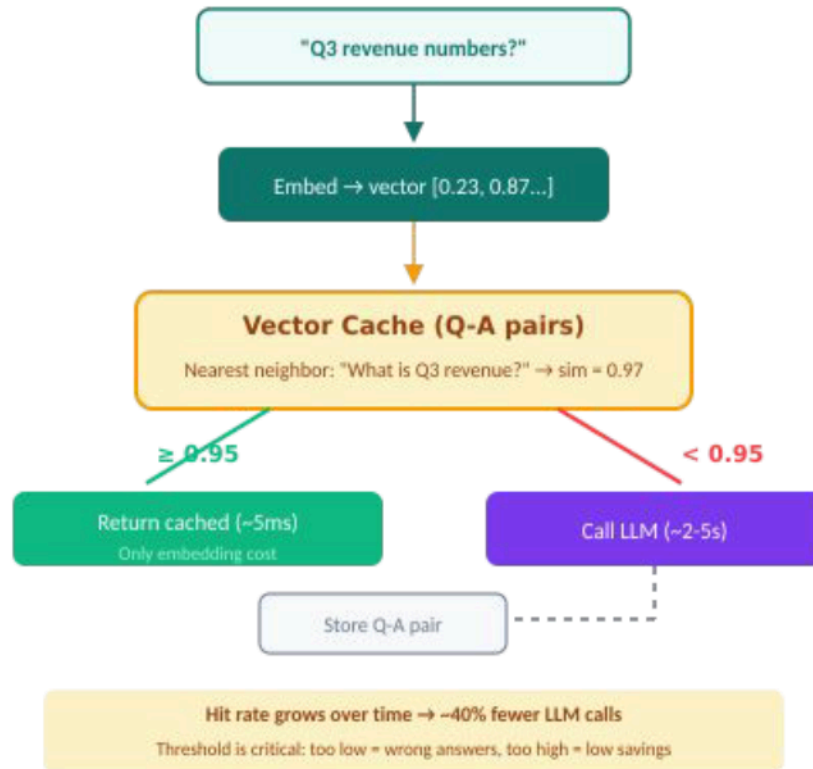
The API provider caches the prefix of your prompt (system message + static docs). On subsequent calls, those cached tokens are processed at a discount — often 50% cheaper. Azure OpenAI API supports this natively.

Why it works:

No code changes needed — just structure your prompt so the static parts come first. The provider handles caching automatically. Saves ~50% on input token costs.

Key requirement:

The cached prefix must be identical across requests. Put the system prompt and reference docs first, the user query last.

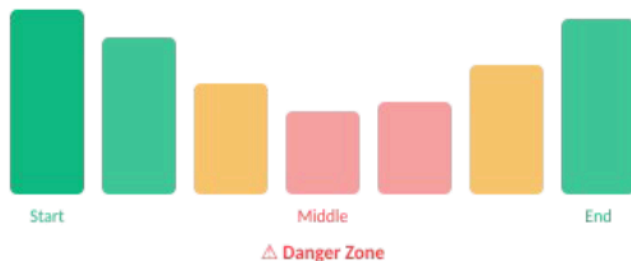




Large Document Handling

Context Window & Lost-in-the-Middle Problem

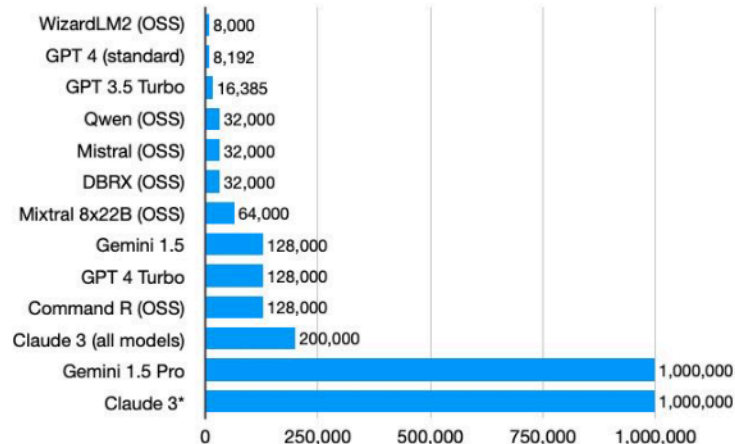
LLM Attention by Position in Context

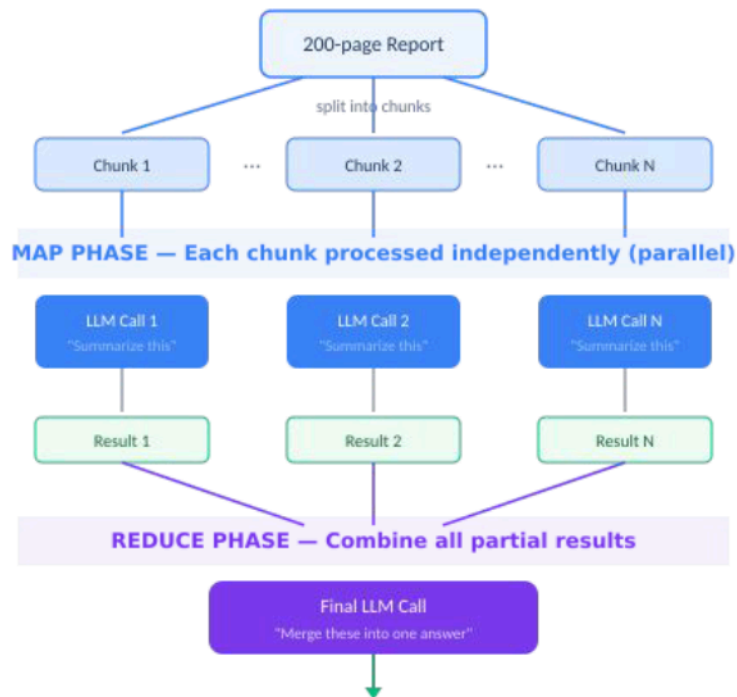


Mitigation Strategies

1. Place most relevant docs at the start and end
2. Reduce total context — fewer, better chunks

Model Context Window Lengths





Large Document Handling

#1 Map-Reduce

The Idea

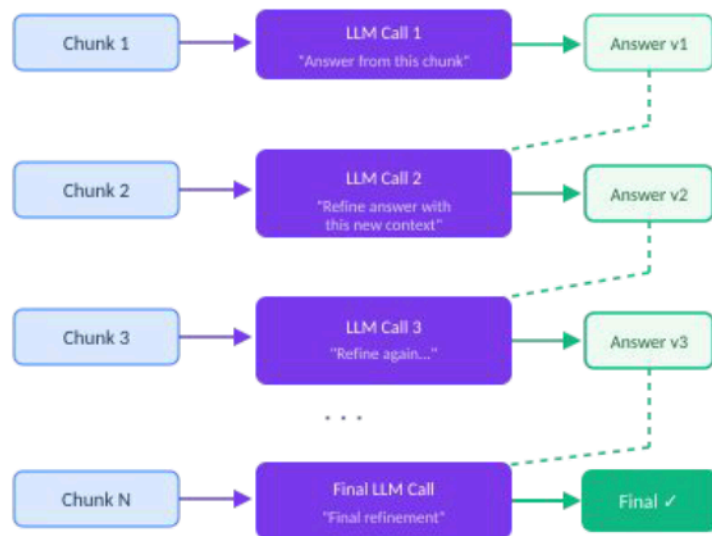
Split the document into chunks. Send each chunk to the LLM independently (MAP phase). Then combine all partial results into one final answer (REDUCE phase).

Why it works:

Each MAP call runs in parallel — a 200-page doc split into 20 chunks takes the same wall-clock time as processing 1 chunk. Massively scalable.

Trade-off:

Each chunk is processed in isolation. Cross-chunk context is lost until the REDUCE step. Good for extraction, less ideal for nuanced reasoning.



Key Difference from Map-Reduce:

Each step sees the previous answer — context accumulates instead of being processed in isolation

Large Document Handling

#2 The Refine Method

The Idea

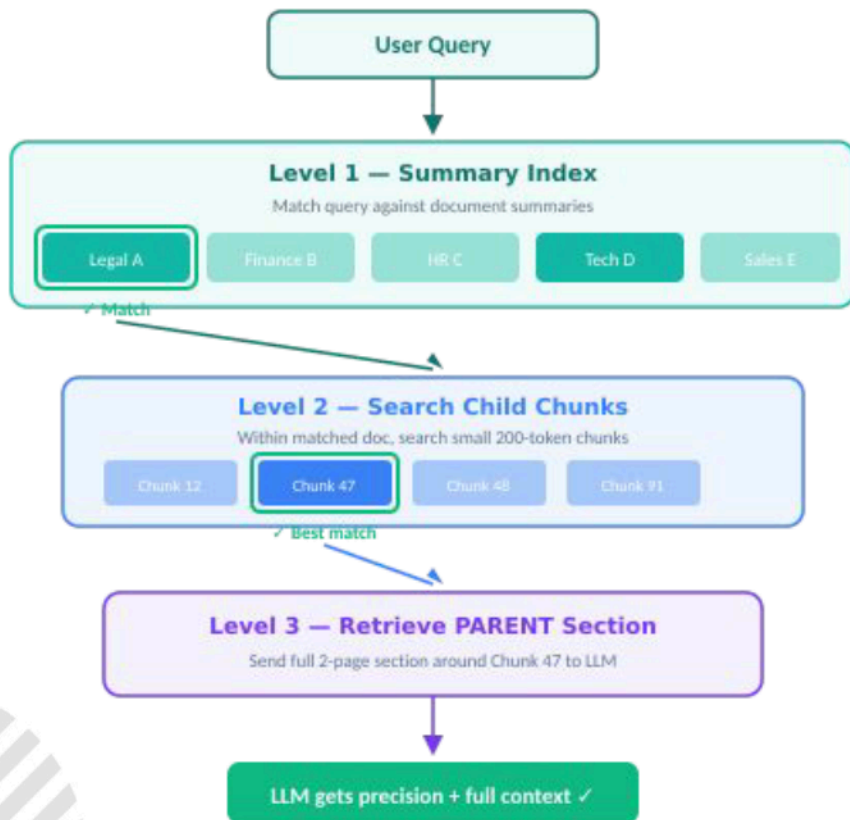
Process chunks sequentially. Start with Chunk 1 to generate an initial answer. Then pass that answer + Chunk 2 to the LLM and ask it to refine. Repeat for every chunk. The answer accumulates context as it goes.

Why it works:

Each step sees the full evolving answer — contradictions get resolved, details get added, nuance is preserved across the entire document.

Trade-off:

Sequential = slow. Can't parallelize. Each step depends on the previous one. Higher total API cost. But the answer quality is significantly better for complex reasoning.



Large Document Handling

#3 Hierarchical Indexing

The Idea

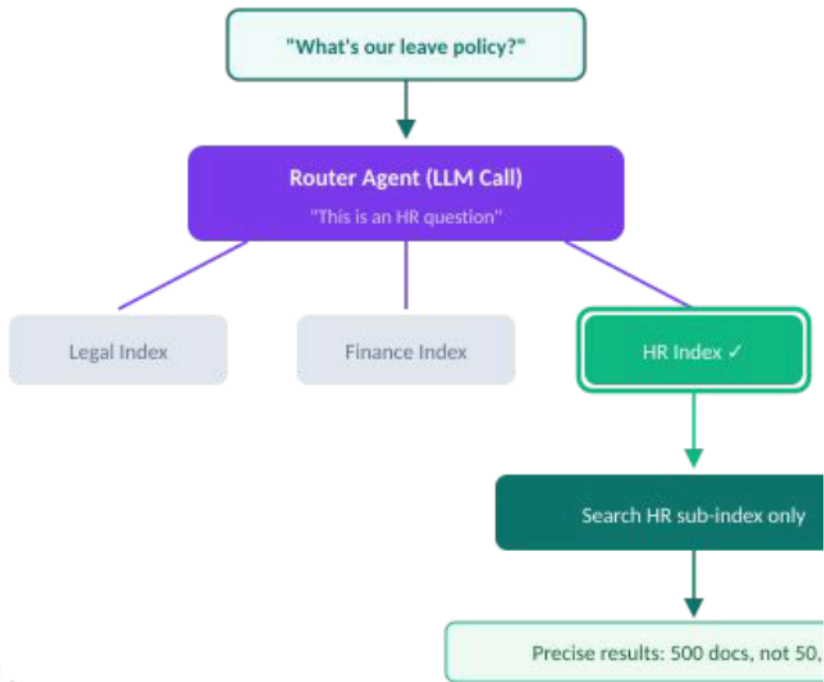
Organize documents in layers. First, match against document summaries (Level 1) to find the right doc. Then search small child chunks inside that doc (Level 2) for precision. Finally, retrieve the parent section around the matched chunk and send that to the LLM (Level 3).

Why it works:

You narrow from millions of docs → a few docs → the right section. The LLM gets the precision of small-chunk matching with the context of the full parent section.

Trade-off:

Requires building and maintaining multiple index layers. More upfront work, but essential at scale.



Large Document Handling

#4 Agentic Routing

The Idea

Use a lightweight LLM call (via Azure OpenAI API) to classify the query intent first. Then route to the correct specialized sub-index before doing any vector search.

Why it works:

Each sub-index is smaller, specialized, and produces better results. The routing call is cheap (a few tokens) compared to searching everything.

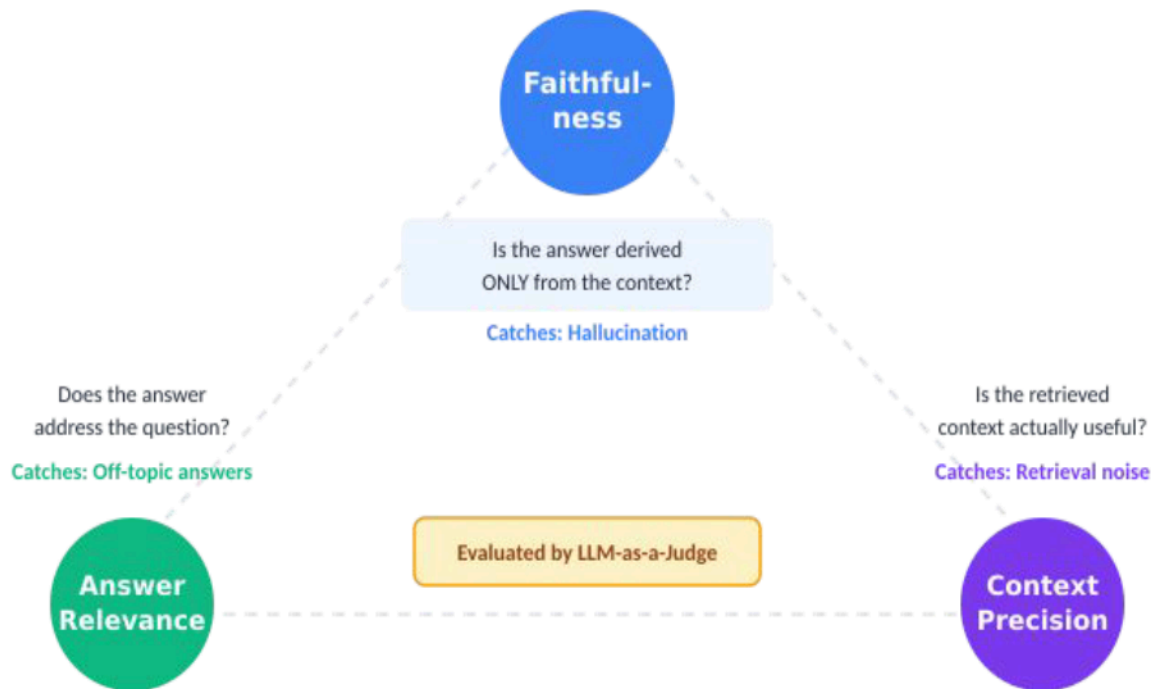
Lost-in-the-Middle warning:

LLMs pay more attention to the start and end of the context window. Place your most relevant chunks there — not buried in the middle of 20 results.



RAG Evaluation Basics

The RAGAS Framework



RAGAS Metric 1: Faithfulness

What it measures

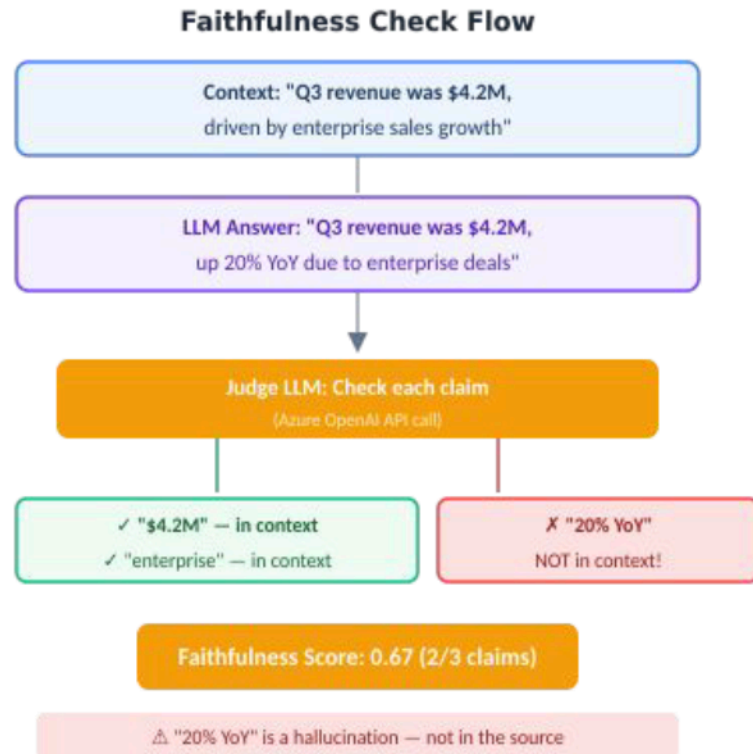
Is the answer derived ONLY from the retrieved context? Or did the LLM inject facts from its training data that aren't in the documents?

Why it matters

This is the hallucination detector. A score of 1.0 means every claim in the answer can be traced back to the context. A low score means the model is making things up.

How to evaluate (LLM-as-a-Judge)

Send the context + answer to a second LLM call via Azure OpenAI API. Ask it: "For each claim in the answer, can you find supporting evidence in the context? Score 0-1."



RAGAS Metric 2: Answer Relevance

What It Measures

Does the answer actually address what the user asked? A perfectly faithful answer can still be useless if it answers a different question.

Why It Matters

Catches off-topic answers. The retrieval may find the right docs, but if the LLM latches onto an irrelevant detail, the answer misses the point entirely.

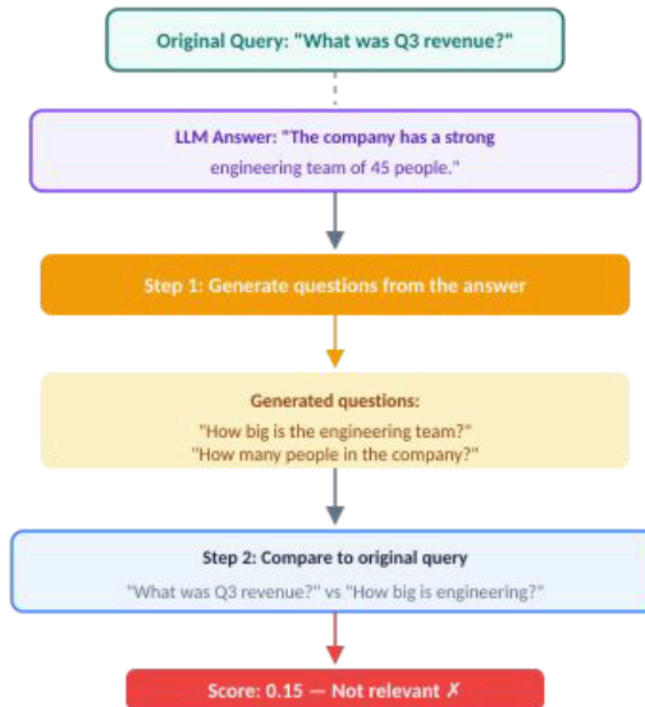
How to Evaluate

Generate hypothetical questions from the answer. If those questions are semantically similar to the original query, the answer is relevant.

Formula:

$$\text{Score} = \text{avg cosine_sim}(\text{original_q}, \text{generated_q})$$

Answer Relevance Check Flow



RAGAS Metric 3: Context Precision

What It Measures

Of the chunks your retrieval pipeline returned, how many are actually useful for answering the question? This evaluates your retrieval quality — not the LLM.

Why It Matters

Low precision = noisy context = the LLM has to wade through irrelevant chunks, increasing hallucination risk and wasting tokens. This is where re-ranking has the biggest impact.

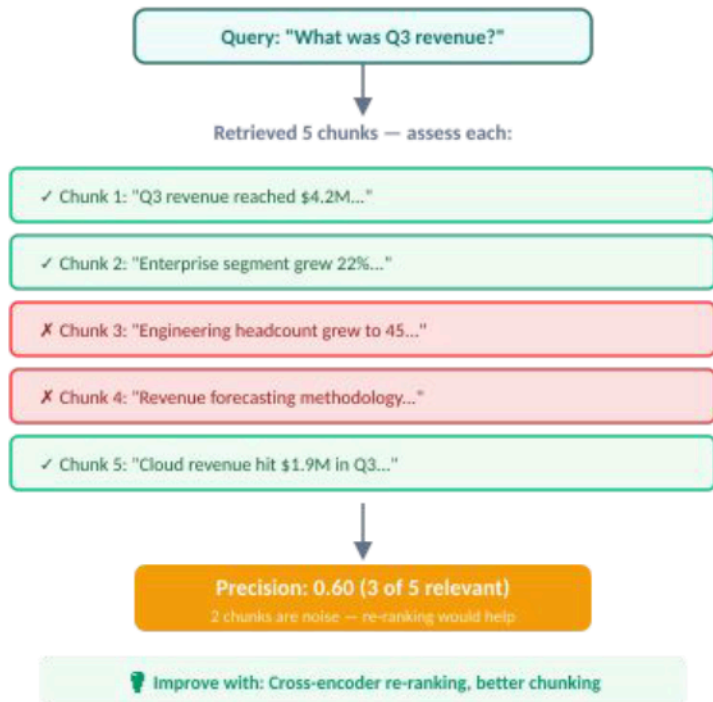
How to Evaluate

For each retrieved chunk, ask the judge LLM: "Does this chunk contain information needed to answer the query?" Then count.

Formula:

Score = # relevant chunks / # total retrieved

Context Precision Check Flow



Evaluation Tools



TruLens

Real-time monitoring dashboard. Tracks cost vs latency vs quality for every RAG request. Alerts when faithfulness drops.

Best for: Production monitoring



Arize Phoenix

Observability platform. Traces your full RAG pipeline, detects embedding drift, enables A/B testing of different retrieval strategies.

Best for: Pipeline debugging



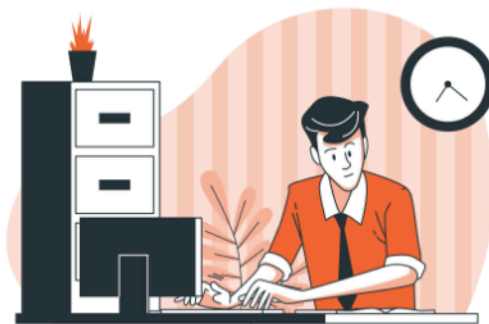
DeepEval

pytest-style unit tests for RAG. 14+ metrics including all RAGAS. CI/CD integration catches regressions before deployment.

Best for: Dev/CI testing

Guided Hands-on

Open in Google Colab [↗](#)



Assignment



Mini RAG Project

Objective

Build a production-grade RAG system from scratch, applying the optimization techniques you learned in the demo notebook. You will work with your own documents and implement retrieval, re-ranking, hallucination reduction, caching, and evaluation.

Scenario

You are an AI engineer tasked with building an internal Q&A system for your organization. The system must answer employee questions accurately using company documents and you must prove it works by evaluating its quality with RAGAS metrics.

[Open in Google Colab ↗](#)



Thank You